Backend Developer – Live Coding Assessment

Duration: 4 Hours | Mode: Live Video Call

Overview

You are required to build a small but realistic microservices-based backend system called **"TaskFlow"** – a simplified project & task management API. The system must be split into three communicating microservices using NestJS, Node.js, PostgreSQL, and RabbitMQ. You may use AI assistants (Claude, ChatGPT, Gemini, etc.) and documentation freely – we are evaluating your ability to architect, implement, debug, and explain your decisions, not your ability to memorize syntax.

Tech Stack (Mandatory)

- Node.js (LTS version)
- NestJS framework
- Microservices architecture (at least 3 services: User, Task, Notification - communicating via RabbitMQ)
- PostgreSQL as the primary database
- TypeORM for DB access (mandatory) with migrations (no synchronize: true) and use of DB transactions where applicable
- JWT-based Auth Guard with role-based access control (RBAC) on protected endpoints (mandatory)
- Basic input validation (class-validator/DTOs)
- Swagger/OpenAPI documentation for both services (mandatory)
- Git for version control (commit progressively, not one final commit)

System Requirements

Service 1: User Service

- Expose REST endpoints to: Register a user (with role: ADMIN or MEMBER), Login (returns access + refresh JWT), Refresh token, Get user by ID, List all users (paginated)
- Store users in PostgreSQL (users table) with role column
- Passwords must be hashed (bcrypt or similar) – plain text not acceptable
- Login issues a short-lived access JWT (containing userId, email, role) and a longer-lived refresh JWT, used by Task Service for authorization & RBAC

Service 2: Task Service

- Expose REST endpoints to: Create a task, Get all tasks (with pagination & filter by status/assignedUserId), Get task by ID, Update task status, Delete a task
- Store tasks in PostgreSQL (tasks table)
- On task creation, the Task Service must call the User Service (via RabbitMQ request/response or TCP) to verify that 'assignedUserId' exists before saving the task. If user doesn't exist, return a 400 error.
- All endpoints protected by JWT Auth Guard; Delete task and reassigning a task must be restricted to ADMIN role only (RBAC guard) – MEMBER attempting these gets 403

- On task status update to DONE, publish an event ('task.completed') to RabbitMQ for the Notification Service to consume
- Wrap task creation + user-verification flow in a way that handles partial failures gracefully (e.g. no orphan records if verification fails)

Service 3: Notification Service

- Consumes 'task.completed' events from RabbitMQ
- On receiving an event, store a notification record in PostgreSQL (notifications table: userId, message, createdAt, isRead)
- Expose a JWT-protected GET /notifications endpoint returning the logged-in user's notifications (paginated)

API Endpoints to Implement

User Service

Method	Endpoint	Description	Auth
POST	/users/register	Register new user (name, email, password, role)	Public
POST	/users/login	Validate credentials, return access + refresh JWT	Public
POST	/users/refresh	Get new access token using refresh token	Public
GET	/users/:id	Get user details by ID	JWT
GET	/users	List all users (paginated)	JWT

Task Service

Method	Endpoint	Description	Auth
POST	/tasks	Create task (validates assignedUserId via User Service)	JWT
GET	/tasks	List tasks - paginated, filter by status/assignedUserId	JWT
GET	/tasks/:id	Get task by ID	JWT
PATCH	/tasks/:id/status	Update task status (TODO/IN_PROGRESS/DONE)	JWT
PATCH	/tasks/:id/assign	Reassign task to another user	JWT (Admin)
DELETE	/tasks/:id	Delete a task	JWT (Admin)

Notification Service

Method	Endpoint	Description	Auth
GET	/notifications	List logged-in user's notifications (paginated)	JWT

Cross-Cutting Requirements

- Use DTOs with validation (e.g. email format, required fields, status enum: TODO/IN_PROGRESS/DONE)
- Use a global exception filter for consistent error responses

- Use environment variables (.env) for DB credentials and service ports/configs
- Add basic logging (NestJS Logger) for incoming requests and inter-service calls
- Write at least 2-3 unit tests (any module of your choice) using Jest

Evaluation Criteria

Criteria	Weight
Code structure & NestJS best practices (modules, services, controllers)	15%
Correct microservices/RabbitMQ communication (sync verification + async events)	20%
Database design, TypeORM migrations & PostgreSQL integration	15%
JWT auth, refresh tokens & RBAC implementation	15%
Validation, error handling, logging & Swagger docs	10%
Problem-solving approach & use of AI tools effectively	10%
Testing (unit tests)	5%
Communication & explanation during walkthrough	10%

Deliverables

- GitHub/GitLab repo link with commit history (or zipped project with .git folder)
- README.md with setup instructions (how to run both services + DB)
- Brief notes on architectural decisions & trade-offs (can be in README)

Rules & Notes

- AI tools (Claude, ChatGPT, Gemini, Copilot) are allowed and encouraged – we want to see how you use them effectively.
- Internet/documentation access is allowed.
- Screen must remain shared throughout the session.
- It's okay if everything isn't 100% complete – partial, well-structured, working code is valued over a rushed full solution.
- Be ready to explain your code and decisions at the end of the session.